

Laboratory 3

Víctor Brull Rico

Guillem Martínez Sambola

May 14, 2026

1 Introduction

Exemplar-based methods are a family of image processing techniques that reconstruct missing or desired content by copying and adapting samples from a known reference region. Two important applications of this idea are texture synthesis, which generates new texture images from a given sample, and image inpainting, which fills in damaged or unwanted regions of a photograph. Despite their different goals, both tasks rely on the same core principle: find a patch in the known area that matches the local context of the region to be filled, and use it to synthesize new content.

This report covers both applications. In Part 1, we implement and test the texture synthesis algorithm of Efros and Leung [1], a foundational method that grows new texture pixel by pixel using non-parametric sampling. In Part 2, we use the IPOL online demo to explore exemplar-based image inpainting and study how the method’s parameters affect the quality of the result.

2 Methods

2.1 Texture Synthesis by Non-parametric Sampling

The method proposed by Efros and Leung [1] treats texture as a Markov Random Field (MRF). This means that the value of any pixel p is assumed to depend only on its local neighborhood $\omega(p)$, a square window of width w centered at p , and not on any other pixel in the image. This locality assumption is the key idea behind the algorithm, in order to synthesize a new pixel, we only need to look at the pixels already synthesized around it.

The synthesis starts from a small seed, typically a 3×3 patch randomly taken from the input texture. From there, the algorithm grows the output image outward, one pixel at a time, following the boundary of the already-synthesized region. For each new pixel, the algorithm searches the input texture for all patches whose known neighborhood is similar to the current context. It then picks one of these candidate patches at random and copies its center pixel to the output.

Similarity between two patches is measured using a Gaussian-weighted sum of squared differences (SSD):

$$d(\omega_1, \omega_2) = \sum_i G_i \cdot (\omega_{1,i} - \omega_{2,i})^2, \quad (1)$$

where G is a 2D isotropic Gaussian kernel. The Gaussian weighting means that pixels near the center of the window contribute more to the distance than those at the edges, which helps preserve local structure. Since some pixels in the neighborhood may not yet be synthesized, only the known pixels participate in the sum, and the result is normalized by the total weight of those pixels so that distances remain comparable.

The candidate patches are those whose distance to the query is within a factor $(1 + \epsilon)$ of the best match found:

$$d(\omega(p), \omega) < (1 + \epsilon) d(\omega(p), \omega_{\text{best}}). \quad (2)$$

The tolerance ϵ controls how random the synthesis is. A small ϵ accepts only very close matches, producing output that closely follows the input structure. A large ϵ accepts many candidates, introducing more variation and stochasticity. The window size w is the other main parameter: it defines how much context is used for matching. Larger windows capture longer-range structure, while smaller windows focus only on local color and texture.

2.2 Implementation Details

The implementation consists of two core functions. `extract_patches.m` slides a $w \times w$ window over the entire sample image and collects every patch whose support lies fully within the image boundaries. Each patch is stored in a 4D array of size (w, w, C, N) , where C is the number of color channels and N is the total number of extracted patches. The center coordinates of each patch are stored in a corresponding $N \times 2$ matrix, in the same order.

`compute_patch_distances.m` computes the Gaussian-weighted SSD between a query patch and all patches in the list at once. A binary mask identifies which pixels in the query are already known; this mask is multiplied element-wise with the Gaussian kernel to form a combined weight map. The weighted squared differences are then summed over height, width, and channels, and normalized by the total weight of the known pixels. The entire computation is vectorized over all N patches simultaneously, which gives a significant speedup compared to a loop-based approach.

2.3 Variational Image In-painting

In the second part of the laboratory, we utilize a variational framework that generalizes exemplar-based matching. Unlike the pixel-by-pixel growth of Part 1, this framework optimizes the entire in-painting domain by minimizing an energy functional. We analyze three specific numerical methods:

- **nlmeans (Non-Local Means):** This method calculates the value of a missing pixel as a weighted average of pixels from similar patches throughout the image. It uses the L_2 -norm for distance, which tends to produce smooth results ideal for stochastic or blurry backgrounds.
- **nlmedians (Non-Local Medians):** Based on the L_1 -norm, this method selects the median value among candidate patches. It is more robust to outliers than the mean, allowing it to preserve sharper edges and avoid the "smudging" effect often seen in L_2 approaches.
- **nlpoisson (Non-Local Poisson):** This approach combines patch matching with a diffusion process by solving a Poisson equation. Instead of copying raw color values, it propagates the gradients (changes in intensity) of the patches. This allows the algorithm to maintain structural continuity and adapt to lighting changes across the in-painting boundary.

3 Experiments

3.1 Texture Synthesis Results

3.1.1 Effect of Patch Size

We first study how the window size w affects the quality of the synthesized texture, keeping the tolerance fixed at $\epsilon = 0.1$. We test four values of w on two textures: bricks (a structured, periodic texture) and cells (a semi-structured organic texture). For each result, we also show the copy map, which encodes the position in the sample image from which each pixel was copied. Regions of similar color in the copy map indicate that consecutive output pixels were taken from nearby locations in the sample.

3.1.2 Bricks

Figure 1 shows the bricks sample and its color map.

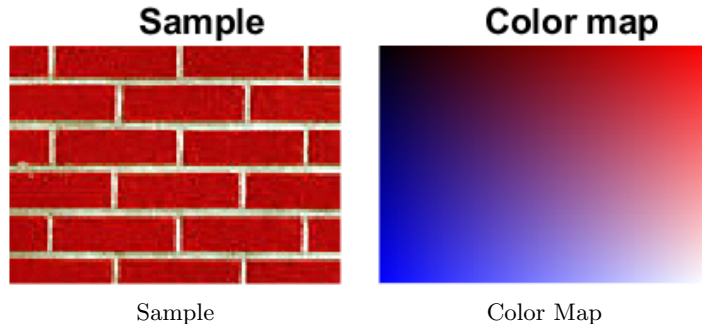


Figure 1: Bricks sample and its color map.

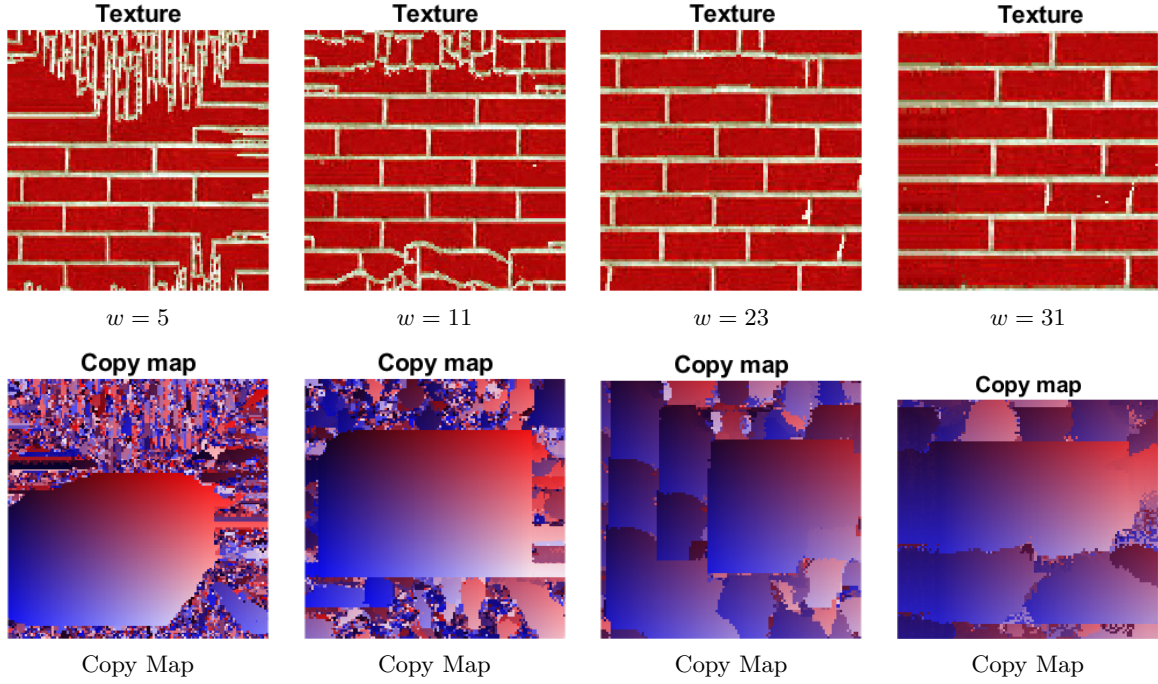


Figure 2: Bricks texture synthesis for increasing window size ($\epsilon = 0.1$). Each column shows the synthesized texture (top) and its copy map (bottom).

With $w = 5$, the window is too small to capture the brick structure. The algorithm can match local color well, but has no information about the broader layout of bricks and mortar. As a result, the output looks noisy and the brick boundaries are misaligned or broken. The copy map is highly scattered, meaning pixels are copied from many different and distant locations in the sample.

As w increases to 11 and then to 23, the synthesized texture improves noticeably. With $w = 23$, the bricks and mortar lines are well preserved and the output is visually convincing. The copy map becomes more coherent, with large smooth regions indicating that consecutive pixels are copied from nearby areas of the sample. This is consistent with the algorithm finding consistent, long-range matches.

At $w = 31$ the result is similar to $w = 23$, but computation time increases substantially. Since the improvement is marginal, $w = 23$ appears to be the best trade-off for this texture.

3.1.3 Cells

Figure 3 shows the cells sample and its color map.

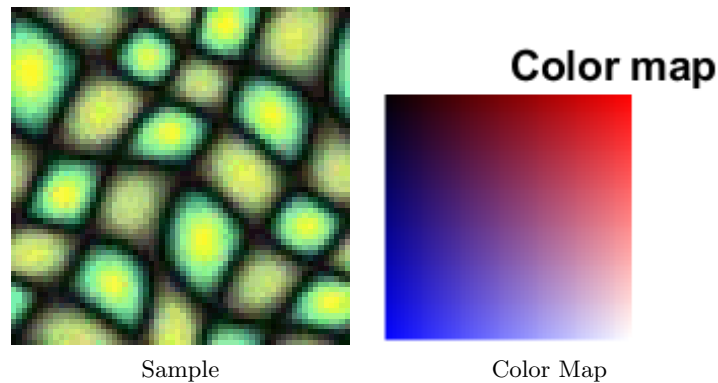


Figure 3: Cells sample and its color map.

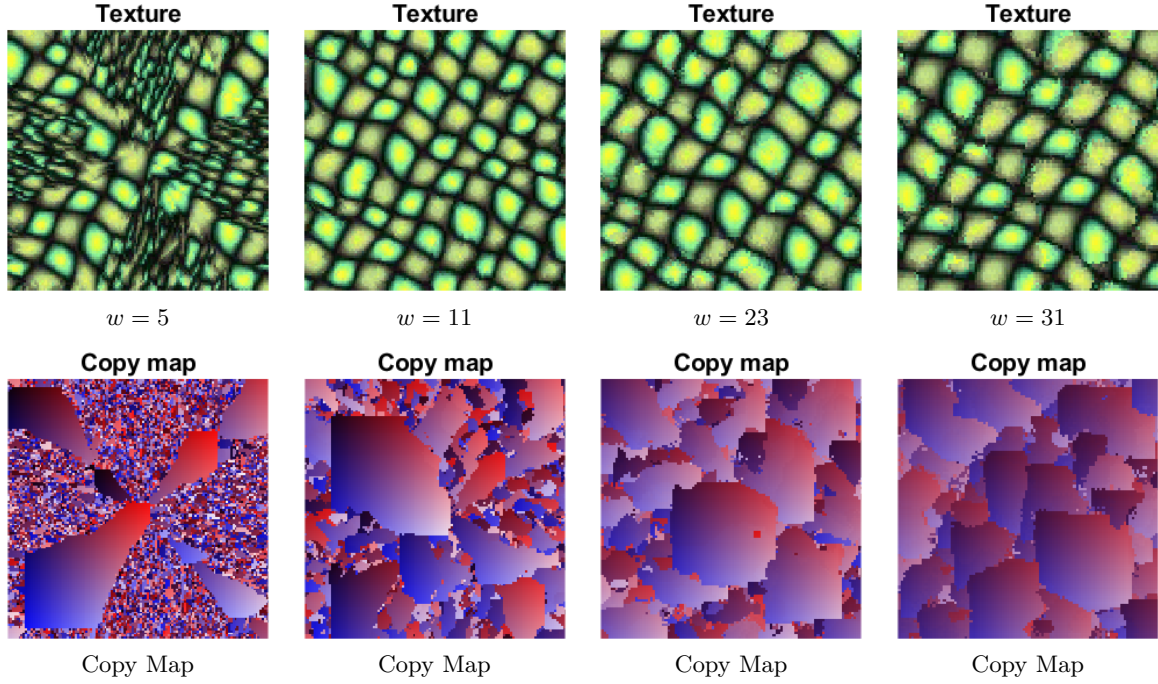


Figure 4: Cells texture synthesis for increasing window size ($\epsilon = 0.1$). Each column shows the synthesized texture (top) and its copy map (bottom).

The cells texture tells a similar story. At $w = 5$, individual cell shapes are deformed and the boundaries between cells appear blurry and incoherent. The copy map is fragmented, showing that the algorithm jumps between many different parts of the sample without consistency.

With $w = 11$ the cell shapes begin to emerge, though some distortion remains at boundaries. At $w = 23$, the synthesized cells are clearly defined and visually similar to the input, with correct colors and shapes. The copy map shows large, smooth regions, indicating that the algorithm is copying from consistent areas of the sample. At $w = 31$ the result remains good but offers no significant improvement over $w = 23$.

These results confirm the observation of Efros and Leung [1], the window size should be chosen to cover at least one full texture element (texel). When the window is smaller than the dominant structure, the algorithm cannot match context correctly and the output degrades.

3.1.4 Effect of Tolerance

We now fix the window size at $w = 23$, which gave the best results in the previous experiment, and vary the tolerance $\epsilon \in \{0.01, 0.1, 1.0\}$.

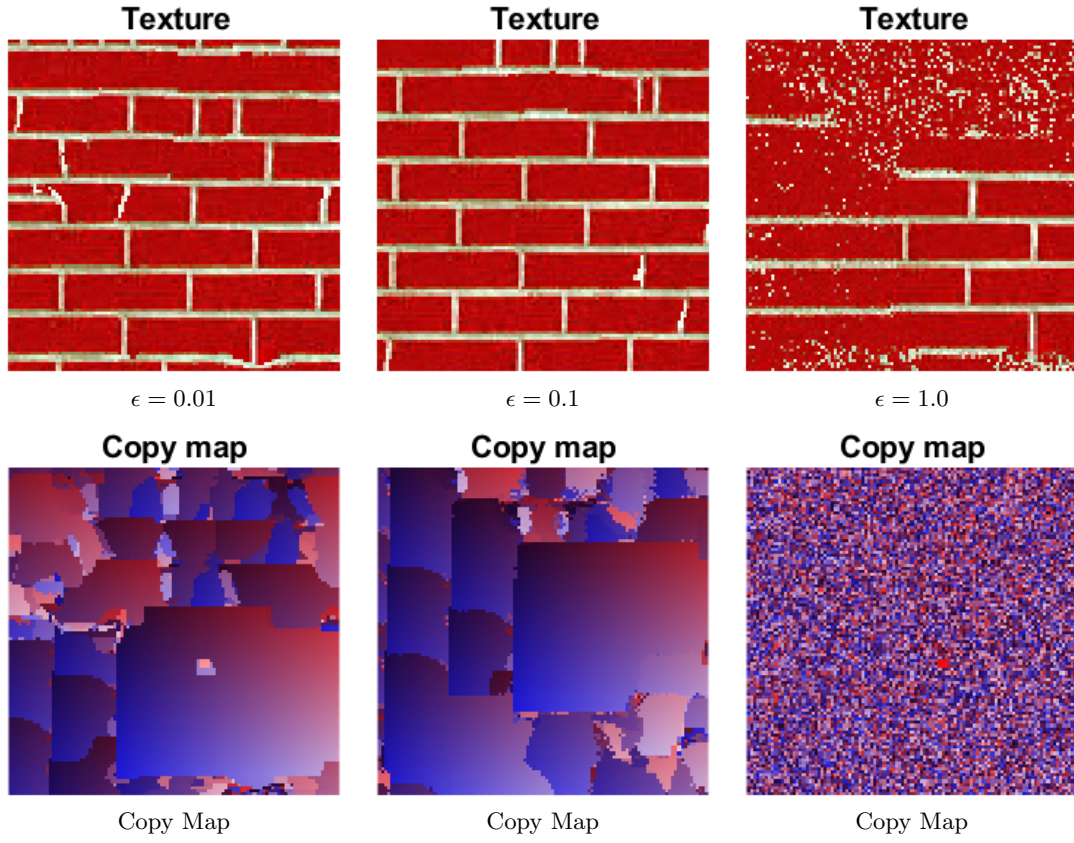


Figure 5: Bricks synthesis for increasing tolerance ($w = 23$). Each column shows the synthesized texture (top) and its copy map (bottom).

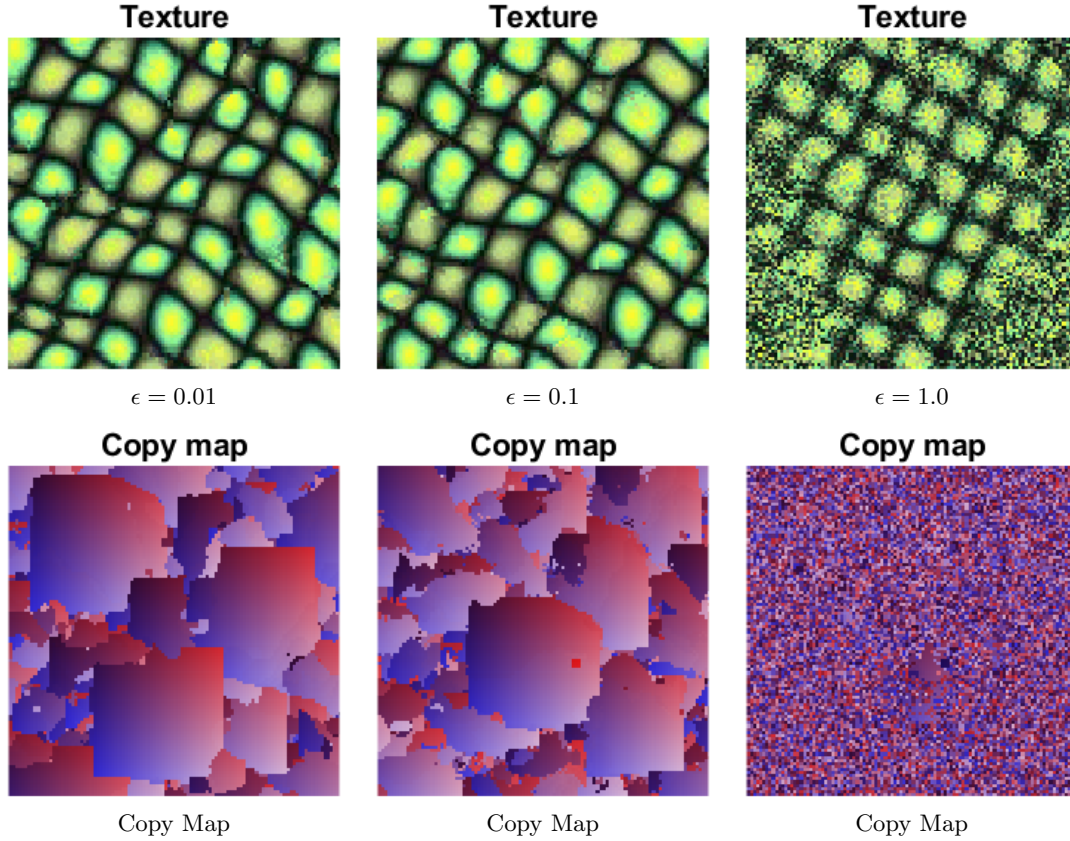


Figure 6: Cells synthesis for increasing tolerance ($w = 23$). Each column shows the synthesized texture (top) and its copy map (bottom).

At $\epsilon = 0.01$, the algorithm accepts only patches that are very close to the best match. This makes the synthesis more deterministic and the output closely follows the spatial layout of the sample. The copy map shows very large, smooth regions, meaning the algorithm tends to copy long consecutive runs from the same part of the sample. For bricks this produces a plausible texture, but the output can look overly regular and less natural because the algorithm has little freedom to vary.

At $\epsilon = 0.1$, more candidates are accepted, and the algorithm can choose from a wider set of matching patches. This introduces natural variation and the output looks more realistic for both textures. The copy map regions are somewhat smaller, reflecting the increased diversity of sources.

At $\epsilon = 1.0$, the tolerance is so large that almost any patch qualifies as a candidate, regardless of how well it actually matches the context. The algorithm effectively samples at random from the input, producing an output that loses the structural coherence of the texture entirely. The copy map becomes very fine-grained and scattered. This is a clear failure case because the output for bricks shows corrupted mortar lines and displaced bricks, while the cells texture becomes unrecognizable.

These experiments show that $\epsilon = 0.1$ provides the best balance between faithfulness to the input structure and natural variation in the output.

3.2 Image In-painting Results

3.2.1 Custom In-painting Results

We used a custom image and a hand-made binary mask to test the restoration capabilities of the variational framework.

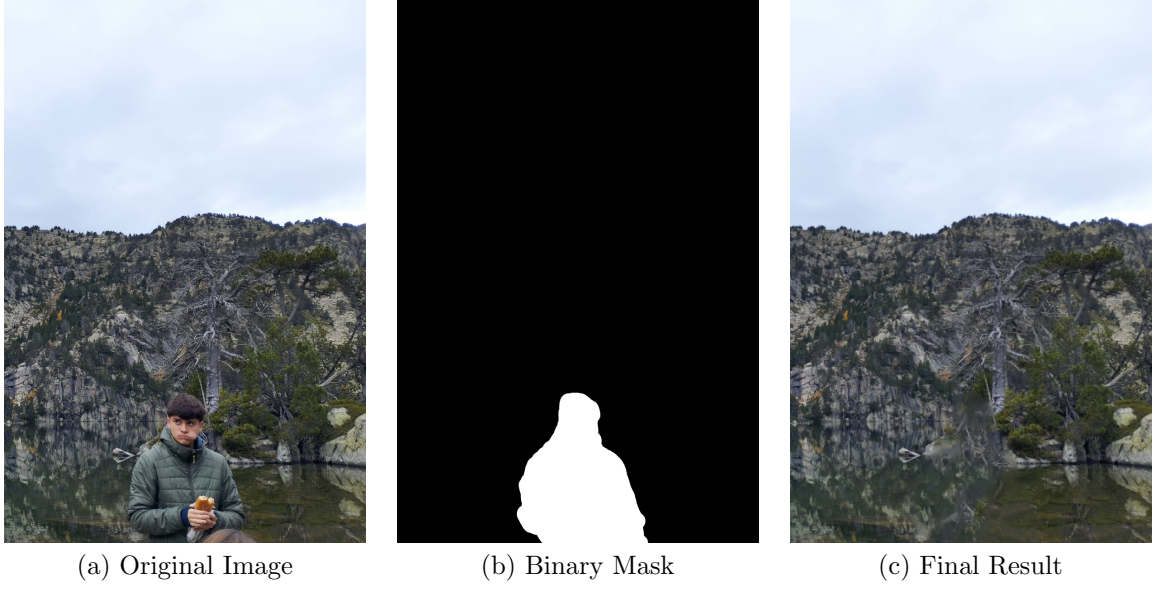


Figure 7: Custom in-painting experiment results.

3.2.2 Parameter Tuning Analysis (12 Configurations)

We systematically explored combinations of `patch_size` and scales (S) across the three numerical methods.

Table 1: Method: **nlmeans** - Scales and Patch Size combinations.

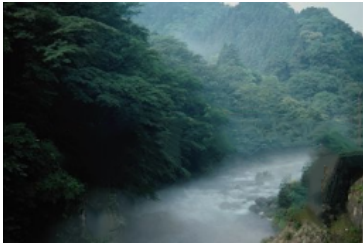
	Min Scale (S=1)	Max Scale (S=20)
Min Patch (5)		
Max Patch (15)		

Table 2: Method: **nlmedians** - Scales and Patch Size combinations.

	Min Scale ($S=1$)	Max Scale ($S=20$)
Min Patch (5)		
Max Patch (15)		

Table 3: Method: **nlpoisson** - Scales and Patch Size combinations.

	Min Scale ($S=1$)	Max Scale ($S=20$)
Min Patch (5)		
Max Patch (15)		

Summary of Parameter Influence:

- **Patch Size:** Smaller patches are ideal for fine, random textures but fail on geometric lines. Larger patches preserve edges but may introduce blurring if no perfect match is found.
- **Scales (S):** Controls multi-resolution processing. Low scales ($S = 1$) are faster but often ignore the global context. High scales ($S \geq 3$) use a Gaussian pyramid to ensure large-scale features remain consistent.
- **Numerical Methods:**
 - *nlmeans*: Uses L_2 averaging for smooth blending; prone to smudging textures.
 - *nlmedians*: Uses L_1 medians for robust, sharper transitions; ideal for high-contrast edges.
 - *nlpoisson*: Uses gradient-based diffusion; provides superior structural flow and adapts best to lighting changes.

4 Conclusion

This laboratory provided a comprehensive exploration of exemplar-based methods, demonstrating their versatility in both texture synthesis and image restoration.

In Part 1, our implementation of the Efros and Leung algorithm highlighted the critical role of the Markov Random Field model. We found that the `patch_size` must be tailored to the size of the texture’s ”primitives” to avoid noisy outputs, while the tolerance parameter ϵ acts as a vital control for balancing structural faithfulness with natural stochastic variation. Our experiments confirmed that a tolerance of $\epsilon = 0.1$ generally provides the most realistic results, preventing both overly deterministic repetition and complete structural collapse.

In Part 2, the evaluation of the IPOL variational framework revealed the distinct trade-offs between different numerical approaches. Our systematic study of 12 configurations showed that:

- **Numerical Methods:** While *nlmeans* excels at smooth blending in stochastic regions, *nlpoisson* is superior for structural propagation as it effectively handles image gradients and lighting consistency.
- **Global Consistency:** High multi-resolution scales (S) are essential for maintaining global coherence (e.g., aligning horizons), ensuring that local patch matching does not conflict with the ”big picture.”
- **Structural Integrity:** Consistent with Part 1, larger patches are necessary to bridge significant gaps and preserve geometric continuity, such as straight lines and sharp edges.

References

- [1] Alexei A. Efros and Thomas K. Leung, ”Texture synthesis by non-parametric sampling,” in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, Corfu, Greece, September 1999, pp. 1033–1038.